

Fundamentals of JS (Tokens)

1. Tokens:

- a. Tokens are the smallest unit of any programming language that can be understood by Javascript.
- b. When Javascript is interpreted, the browser parses the script into these tokens.
- c. A tokenizer is a fundamental component in natural language processing and parsing tasks.
- d. It breaks down a string of characters or words into smaller units, called tokens or the process of breaking fundamental components of language into tokens is known as tokenizer.
- e. **JavaScript tokens fit in 6 categories:**
 - i. Keywords
 - ii. Identifiers
 - iii. Literals
 - iv. Operators
 - v. Separators
 - vi. Comments

2. Keywords:

- a. Keywords are predefined reserved words in any programming language.
- b. Each keyword has special meaning internally.
- c. It is always written in lowercase.
- d. **NOTE** ⇒ The programmer cannot modify the assigned task of the keyword.

e. Here is the list of reserved keywords:

abstract	double	implements	return
arguments	else	in	switch
await	enum	instanceof	synchronized
boolean	eval	int	this
break	export	interface	throw
byte	extends	let	throws
case	false	long	transient
catch	final	native	true
char	finally	new	try
class	float	null	typeof
const	for	package	var
continue	function	private	void
debugger	goto	protected	volatile
default	if	public	yield
delete	implements	short	while
do	import	static	with
double	in	super	

f. Reserved Keywords added in ES5 and ES6:

- i. Some new keywords are added in the ES5 and ES6 versions of JavaScript. However, some are currently in use, and some keywords are reserved for future versions.

await	class	enum	export
extends	import	let	Super

3. Identifiers:

- a. Identifiers are simply names that represent variables, function or property.
- b. They consist of a combination of characters and digits.
- c. It is usually defined by the user/programmer.

d. Rules to define identifier:

- i. **Rule-1** → The first letter of an identifier must be a letter, underscore (`_`) or a dollar sign (`$`).
- ii. **Rule-2** → The whitespace cannot be included in the identifier.
- iii. **Rule-3** → Identifiers are case sensitive.
- iv. **Rule-4** → Letters include all uppercase characters, "**A-Z**", and all lowercase characters, "**a-z**".
- v. **Rule-5** → Digits include the characters **0-9**.

e. Identifier Naming Conventions:

- i. Camel Case Convention →

Example → AMARAKABRANOTHONY =

amarAkbarAnothony → Camel Case

NOTE: *Camel case convention which is used to provide an identifier to function, variable and method, etc.*

4. Literals:

- a. Literals are data used to represent fixed values in JavaScript.

b. Integer Literals:

- i. These are whole numbers. You can write them in:
- ii. **Decimal (base 10):** e.g. **33**, **56789**
- iii. **Octal (base 8):** e.g. **071**, **0566**

c. Floating-Point (Decimal) Literals:

- i. These are numbers with decimal points or written in scientific notation:
- ii. Examples: `3405.673`, `-1.958`, `8.32`, `9.98`

d. Boolean Literals:

- i. Only two values:
- ii. `true` (means yes / 1)
- iii. `false` (means no / 0)

e. Object Literals:

- i. Used to create objects with key-value pairs:
- ii. Example: `{ name: "John", age: 30 }`

f. Array Literals:

- i. Used to create arrays (lists of values):
- ii. Example: `[1, 2, 3, 5]`

g. Null Literal:

- i. Represents **no value** or **empty value**:
- ii. Example: `null`

5. Comments:

- a. The JS comments are a meaningful way to deliver messages.
- b. It is used to add information about the code, warnings or suggestions. So that a user can easily interpret the code.
- c. They are ignored by the interpreter but are useful for documenting the code.

d. Types of JS Comments →

- i. Single-Line Comment: *// This is a single line comment.*
- ii. Multi-Line Comment: */* This is a multi-line comment */*

6. Operators:

- a. Symbols that perform operations on operands.
- b. Example → +, -, %, <, > etc

7. Separators:

- a. Characters that have syntactic meaning.
- b. Example →
 - i. Semicolons: ;
 - ii. Comma: ,
 - iii. Parentheses: ()
 - iv. Braces: {}, used for blocks of code or object literals
 - v. Brackets: [], used for arrays or indexing